Результаты проверки текста на оригинальность

Plagiat.AI

Оригинальность	Заимствования
75%	25%
Цитирование	**Самоцитирование**
0%	0%

Проверка на ИИ: Оценка вероятности написания текста человеком / сгенерированного нейросетью

Человек: 98% ИИ: 2%

Список источников:	**Модуль**
https://files.stroyinf.ru/Data1/9/9439/index.htm	Интернет Free
https://companies.rbc.ru/id/1027807985094-federalnoe-gosudarstvennoe-byudzhetnoe-obrazovatelnoe-uchrezhdenie-vyisshego-obrazovaniya-sankt-peterburgskij-gosudarstvennyij-institut-kino-i-televideniya/	Интернет Free
https://zkrf.ru/tk/CHAST-CHETVERTAYA/Razdel-XII/Glava-55/Statya-351/	Интернет Free
https://www.pnp.ru/politics/zakony-vstupayushhie-v-silu-s-14-noyabrya-2.html	Интернет Free
https://rcbutovo.mos.ru/legislation/rehabilitation-and-habilitation/Федеральный закон от 24.11.1995 N 181-ФЗ (ред. от 26.05.2021.pdf	Интернет Free
https://www.disright.org/ru/info/adaptaciya-invalida-k-trudovoy-deyatelnosti	Интернет Free
http://publication.pravo.gov.ru/Document/View/0001202111060007	Интернет Free
https://its.1c.ru/db/content/newscomm/src/475534.htm	Интернет Free
https://ktzszmoik.gov.by/adaptaciya-invalidow/	Интернет Free
https://www.miacugra.ru/poleznaya-informatsiya/o-kvotirovanii-rabochikh-mest-invalidam.php	Интернет Free

Текст:

МИНОБРНАУКИ РОССИИ Федеральное государственное бюджетное образовательное учреждение высшего образования «МИРЭА – Российский технологический университет» РТУ МИРЭА Институт информационных технологий ИТ Кафедра инструментального и прикладного программного обеспечения ИиППО КУРСОВАЯ РАБОТА по дисциплине: Бэкенд-разработка по профилю: Разработка программных продуктов и проектирование информационных систем направления профессиональной подготовки: 09.03.04 «Программная инженерия» Тема: «Серверная часть веб-приложения для управления образовательными курсами» Студент: Миркин Кирилл Леонидович Группа: ИКБО-10-23 Работа представлена к защите ________дата________/Миркин К.Л./ подпись и ф.и.о. студента Руководитель: ст. преподаватель Волков Михаил Юрьевич Работа допущена к защите ________дата____________/Волков М.Ю./ подпись и ф.и.о. рук-ля Оценка по итогам защиты: __________ _______________ / ст. преподаватель Волков М.Ю. / _______________ / / М. РТУ МИРЭА. 2026 Заведующему кафедрой инструментального и прикладного программного обеспечения ИиППО Института информационных технологий ИТ Болбакову Роману Геннадьевичу От студента Миркина Кирилла Леонидовича группы ИКБО-10-23 3 курса Контакт: mirkin.k.l@edu.mirea.ru Заявление Прошу утвердить мне тему курсовой работы по дисциплине «Бэкенд-разработка» образовательной программы бакалавриата 09.03.04 Программная инженерия Тема: Серверная часть веб-приложения для управления образовательными курсами Приложение: лист задания на КР в 2-ух экземплярах на двустороннем листе проект Подпись студента Миркин К.Л. подпись ФИО Дата 20.02.2026г. Подпись руководителя ст. преп. Волков М.Ю. подпись Должность, ФИО Дата 20.02.2026г. Согласовано Зав. кафедрой ИиППО 20.02.2026г.____________________ / Болбаков Р.Г / РЕФЕРАТ Отчёт 45 с., 9 рис., 5 табл., 26 ист. CLEAN ARCHITECTURE, FASTAPI, REST API, СЕРВЕРНАЯ ЧАСТЬ Миркин К.Л., Курсовая работа направления подготовки «Программная инженерия» предмета «Бэкенд-разработка» на тему «Серверная часть веб-приложения для управления образовательными курсами»: М. 2026 г., МИРЭА – Российский технологический университет РТУ МИРЭА, Институт информационных технологий ИИТ, кафедра инструментального и прикладного программного обеспечения ИиППО Курсовая работа посвящена разработке серверной части веб-приложения для управления образовательными курсами. Выполнен анализ предметной области, сформированы функциональные и нефункциональные требования к системе управления обучением LMS, проведён обзор существующих решений с построением матрицы отслеживаемости аналогов. Проведён анализ архитектурных паттернов DDD, Clean Architecture и MVC, выполнен их сравнительный анализ и обоснован выбор Clean Architecture в качестве основного подхода к проектированию серверной части. Разработана детальная архитектура системы на основе Clean Architecture с описанием слоёв API, сервисов, репозиториев и моделей. Реализована серверная часть на языке Python с использованием фреймворка FastAPI, СУБД PostgreSQL и библиотеки SQLAlchemy. Приведены листинги кода, описана структура REST API, слой базы данных и механизмы аутентификации. Корректность разработанной архитектуры подтверждена реализованным прототипом системы. 4 СОДЕРЖАНИЕ Определения, обозначения и сокращения . . . . 7 Введение . . . . 9 1 Анализ предметной области и требований . . . . 11 1.1 Описание предметной области . . . . 11 1.2 Требования к системе . . . . 11 1.2.1 Функциональные требования . . . . 11 1.2.2 Нефункциональные требования . . . . 12 1.2.3 Ограничения проекта . . . . 13 1.3 Анализ существующих LMS-решений . . . . 13 1.4 Матрица отслеживаемости аналогов . . . . 15 1.5 Итоги анализа предметной области . . . . 16 2 Выбор и обоснование технологий . . . . 17 2.1 Анализ архитектурных паттернов . . . . 17 2.1.1 Domain-Driven Design . . . . 17 2.1.2 Clean Architecture . . . . 17 2.1.3 MVC и MVT в контексте серверной разработки . . . . 18 2.2 Сравнение паттернов и обоснование выбора . . . . 19 2.3 Обоснование технологического стека . . . . 20 2.4 Итоги выбора технологий . . . . 22 3 Разработка архитектуры приложения . . . . 23 3.1 Слои архитектуры . . . . 23 3.2 Dependency Injection как механизм связывания слоёв . . . . 24 3.3 Обработка ошибок . . . . 24 3.4 Диаграмма компонентов . . . . 25 3.5 Проектирование базы данных . . . . 27 3.6 Результаты проектирования . . . . 29 4 Разработка серверной части интернет-ресурса . . . . 30 4.1 Реализация REST API на FastAPI . . . . 30 4.2 Реализация сервисного слоя . . . . 31 4.3 Слой безопасности . . . . 33 4.4 Управление конфигурацией и запуск . . . . 34 4.5 Тестирование . . . . 35 4.6 Клиентское представление . . . . 36 4.7 Итоги разработки . . . . 40 Заключение . . . . 41 Список использованных источников . . . . 43 6 ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ В настоящем отчёте применяют следующие определения, обозначения и сокращения. API Application – программный интерфейс, определяющий Programming Interface способы взаимодействия компонентов программного обеспечения. Argon2 – алгоритм хэширования паролей, устойчивый к атакам с использованием GPU и ASIC. Clean Architecture – архитектурный паттерн, предложенный Робертом Мартином, обеспечивающий независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов. DDD Domain-Driven Design – подход к проектированию программного обеспечения, ориентированный на моделирование предметной области и использование единого языка Ubiquitous Language. Dependency Injection DI – паттерн проектирования, при котором зависимости компонентов передаются извне, а не создаются внутри компонента. DTO Data Transfer – объект передачи данных, используемый для Object обмена структурированной информацией между слоями приложения. ER-диаграмма Entity- – диаграмма «сущность–связь», используемая для Relationship Diagram моделирования структуры данных. FastAPI – современный веб-фреймворк для Python, ориентированный на создание REST API с поддержкой асинхронности, автоматической валидации данных и документирования. HTTP HyperText Transfer – протокол передачи данных между клиентом и Protocol сервером в веб-среде. 7 JWT JSON Web Token – компактный и самодостаточный формат для передачи данных в виде JSON-объекта, используемый для аутентификации. LMS Learning – система управления обучением, Management System предназначенная для администрирования учебного процесса, хранения материалов и контроля результатов. MVC Model–View–Controller – архитектурный паттерн, разделяющий Controller приложение на три взаимосвязанных компонента: модель, представление и контроллер. ORM Object-Relational – технология отображения объектных структур в Mapping реляционные базы данных. Repository – паттерн, инкапсулирующий логику доступа к данным и предоставляющий интерфейс для операций с сущностями. REST Representational – архитектурный стиль взаимодействия State Transfer распределённых систем, использующий принципы унифицированного интерфейса и обмена представлениями ресурсов. Service Layer – слой бизнес-логики приложения, обрабатывающий прикладные операции и обращающийся к репозиториям. SQL Structured Query – язык запросов, используемый для управления Language реляционными базами данных. SQLAlchemy – библиотека ORM и инструмент для работы с реляционными базами данных в Python. СУБД – система управления базами данных. Фреймворк – программный каркас, упрощающий разработку приложений и задающий архитектурные принципы. 8 ВВЕДЕНИЕ Разработка серверной части современных веб-приложений представляет собой одну из ключевых дисциплин в области программной инженерии. Серверная часть backend отвечает за обработку бизнес-логики, хранение и предоставление данных, аутентификацию пользователей, обеспечение безопасности и масштабируемости системы. Качество архитектурных решений, принятых на этапе проектирования серверной части, напрямую определяет такие характеристики итогового продукта, как производительность, надёжность, сопровождаемость и возможность дальнейшего расширения функциональности [1]. Одной из актуальных областей применения backend-разработки являются системы управления обучением Learning Management Systems, LMS [2]. Такие системы обеспечивают организацию учебного процесса в цифровой среде: управление курсами и учебными материалами, проведение тестирования, отслеживание успеваемости и коммуникацию между участниками образовательного процесса. С ростом популярности дистанционного и смешанного форматов обучения возрастают требования к архитектуре подобных систем, что делает задачу проектирования и реализации их серверной части особенно значимой. Актуальность выбранной темы определяется необходимостью разработки серверной части для современной LMS-системы с применением актуальных архитектурных паттернов и технологического стека. Особое внимание уделяется Clean Architecture – подходу, обеспечивающему независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов. Применение данного паттерна позволяет создать систему, устойчивую к изменениям требований и удобную для сопровождения [3]. Цель работы – разработка серверной части веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture с использованием современных технологий и инструментов. Для достижения поставленной цели необходимо решить следующие задачи: 9 – выполнить анализ предметной области и определить требования к серверной части LMS-системы; – провести обзор и сравнительный анализ архитектурных паттернов DDD, Clean Architecture и MVC, применимых к разработке серверной части; – обосновать выбор технологического стека FastAPI, PostgreSQL, SQLAlchemy для реализации серверной части; – разработать архитектуру серверной части на основе Clean Architecture с детальным описанием слоёв; – реализовать серверную часть приложения, включая REST API, слой бизнес-логики и слой доступа к данным; – провести тестирование разработанной системы. Объектом исследования – процессы организации, хранения и предоставления учебных данных. Предметом исследования являются архитектурные и технические решения при разработке серверной части системы управления обучением с применением Clean Architecture. 10 1 Анализ предметной области и требований 1.1 Описание предметной области Система управления онлайн-обучением LMS, Learning Management System – программная платформа, предназначенная для организации, сопровождения и контроля учебного процесса в цифровой среде. Платформа обеспечивает студентам доступ к учебным курсам, материалам и заданиям, а преподавателям – инструменты управления контентом, формирования и проверки заданий, оценки результатов. Предметная область охватывает несколько ключевых компонентов: – учебные курсы с теоретическими материалами, практическими заданиями и тестами; – пользователи с разграниченными ролями: преподаватели и студенты; – механизмы тестирования и оценки знаний, обеспечивающие контроль усвоения материала; – хранилище данных – профили пользователей, курсы, результаты, учебные артефакты. Типичные сценарии использования LMS включают регистрацию и аутентификацию пользователей, управление курсами и уроками, загрузку обучающих материалов, формирование и выдачу заданий, сохранение результатов и формирование отчётности. Структура и функциональные возможности LMS рассмотрены в исследовании Rosário и Dias [2]. Принципы программной инженерии, применяемые при построении подобных систем – модульность, разделение ответственности, изоляция слоёв, – описаны в учебных пособиях по дисциплине [4]. 1.2 Требования к системе 1.2.1 Функциональные требования На основе анализа предметной области сформированы функциональные требования к серверной части системы. 11 Таблица 1 – Функциональные требования Группа Содержание требования Управление курсами и Преподаватель создаёт учебные классы, формирует уроки и управляет их составом. Студенты получают доступ к урокам. Пользователи и роли Регистрация, аутентификация и разграничение прав. Поддержка ролей «преподаватель» и «студент» с возможностью переключения активной роли. База задач и домашние Преподаватель ведёт собственную базу задач и использует их повторно в заданиях разных домашних заданиях. Домашнее задание формируется как набор задач, привязанный к уроку. Тестирование и Студент отправляет ответы; система автоматически проверка сверяет их с эталоном, фиксирует результат, число попыток и затраченное время. Прогресс и статистика Студент видит личный прогресс; преподаватель – агрегированную статистику по классу. Чат класса Обмен сообщениями в реальном времени между участниками класса с сохранением истории. Приглашения в класса Преподаватель генерирует код-приглашение; студент вступает в класс по коду. Теоретические Иерархические теоретические материалы, привязанные к материалы урокам и доступные студентам после публикации. 1.2.2 Нефункциональные требования Нефункциональные требования определяют эксплуатационные характеристики системы и влияют на архитектурные решения [5]. Оформление проектной документации ведётся по ГОСТ 7.32-2017 [6]. Система ориентирована на городскую образовательную среду с числом пользователей до 100 000 человек. 12 Таблица 2 – Нефункциональные требования к системе Критерий Метрика Значение Обоснование Производительность Время отклика 0–1000 мс Отклик до 100 мс воспринимается как мгновенный; до 1000 мс – как естественный. Время загрузки 5–10 с Обеспечивает комфортную загрузку навигацию между уроками и интерфейса заданиями. Нагрузка Пиковая 5 000–10 000 При 100 000 активность пользователей зарегистрированных пользователей пиковая активность – 10% в учебные часы. Интенсивность 500–1000 RPS 10 ~ 000 × 6/60 = 1000 RPS запросов, RPS при 6 вызовах API на одно действие. Надёжность Доступность ≥ 99,5 % Непрерывная доступность в учебные часы и периоды контрольных мероприятий. Совместимость Браузеры Chrome, Поддержка актуальных версий Firefox, Edge, основных браузеров и Safari школьной и домашней среде. 1.2.3 Ограничения проекта Система разрабатывается одним исполнителем в учебные сроки, что ограничивает применение сложных инфраструктурных решений и предопределяет необходимость ориентироваться на ключевые сценарии использования. Архитектура должна оставаться компактной и допускать дальнейшее расширение без переписывания ядра. 1.3 Анализ существующих LMS-решений Для обоснования требований и выявления конкурентных преимуществ разрабатываемой системы рассмотрены четыре широко используемые платформы: Moodle, Google Classroom, Stepik и Фоксфорд [7–10]. Цель анализа – понять, насколько хорошо каждая из них закрывает сформулированные требования, и определить, в каких аспектах разрабатываемое решение может их превзойти. Moodle – наиболее распространённая LMS с открытым исходным кодом более 20 лет на рынке [7]. Написана на PHP и развёртывается на собственном сервере. Сильная сторона Moodle – богатство плагинов и зрелый механизм тестирования, поддерживающий десятки типов вопросов. Слабые стороны: сложность развёртывания и поддержки, устаревший пользовательский интерфейс и отсутствие удобного современного REST API, что затрудняет создание сторонних клиентов и интеграций. Независимая база задач в Moodle реализована лишь частично: вопросы хранятся в категориях, но привязаны к курсам, а не к платформе глобально. Google Classroom – облачная платформа Google, интегрированная с Google Workspace for Education [8]. Ориентирована на максимальную простоту: курс создаётся за несколько минут без установки инфраструктуры. Платформа хорошо подходит для выдачи заданий в формате документов и ссылок на Google Drive, но практически не поддерживает автоматическую проверку ответов – задания проверяются вручную. REST API существует, однако покрывает лишь базовые операции с курсами и заданиями; WebSocket-чат и приглашения по коду отсутствуют. Развернуть платформу на собственном сервере невозможно. Stepik – российская платформа с сильным акцентом на интерактивное тестирование [9]. Stepik поддерживает разнообразные типы задач – строковые, числовые, код на разных языках, выбор из вариантов с полностью автоматической проверкой – это ключевое преимущество перед остальными аналогами. Платформа предоставляет REST API для курсов и задач. Основное ограничение: Stepik ориентирован на публикацию открытых или платных курсов, а не на модель «закрытый класс с преподавателем», которая нужна для разрабатываемой системы. Создание учебных классов, управление домашними заданиями и realtime-чат не являются нативными функциями платформы. Фоксфорд – российская онлайн-школа для учеников 1–11 классов [10]. По функциональной модели Фоксфорд наиболее близок к разрабатываемой системе: есть классы, уроки, домашние задания, чат и статистика успеваемости. Однако это полностью закрытая коммерческая система без публичного API и возможности развернуть у себя. Исходный код закрыт, интеграция с внешними инструментами не предусмотрена. 14 1.4 Матрица функциональных возможностей аналогов Анализ аналогов позволяет сопоставить функциональность платформ и понять, насколько каждая из них соответствует сформулированным требованиям. Для наглядности результаты анализа разделены на две таблицы: функциональные возможности представлены в таблице 3, а архитектурные характеристики сопоставлены в таблице 4. Таблица 3 – Матрица функциональных возможностей LMS-аналогов Google Функция Moodle Stepik Фоксфорд Classroom Курсы и уроки Разделы, Темы с потоком Уроки, видео, Структурно- страницы, заданий, без уроков. дедлайны. материалы, условный Курсы, разбиение на секции, уроки, шаги. доступ. ссылок. задач для выделения курсу. Глобальная база задач Банк вопросов Задания в виде Глобальная база ДЗ без привязки к файловой ссылок, за задач и ДЗ разных курсов. задач и сущность. Автопроверка Quiz-модуль, 15 Ручная Мощный Тесты; типов вопросов. проверка автогрейдер, свободный ввод преподавателем. проверка кода. проверяется вручную. Статистика Отчёты по Список Баллы, Оценки, активности, прогресс, оценки, сданных работ, позволяет, журнал оценки. аналитика по прогресс по доступа. шагам. урокам. Чат realtime Асинхронные Внешний Асинхронные Встроенный чат форумы и Google Chat. комментарии к класса в сообщения. задачам. реальном времени. Приглашения Ключи записи Код приглашения. Ссылка для Запись через Запись на курс. вступления, поиск или управляется ссылку. администра- торам. В таблице 4 приведено сопоставление платформ по инфраструктурным параметрам. 15 Таблица 4 – Матрица архитектурных характеристик LMS-аналогов Google Характеристика Moodle Stepik Фоксфорд Classroom Открытый код Да GPL-3.0. Нет. Нет. Нет. Развёртывание Собственный Облако Google. Облако Stepik. Закрытая SaaS- сервер PHP, система. БД, плагины. REST API Сложный и Ограниченный Полноценный Публичный API устаревший. базовые открыт для отсутствует. операции. разработчиков. Матрицы показывают, что у каждой платформы есть свои ниши: Moodle силён в тестировании, Stepik – в автопроверке задач, Google Classroom – в простоте. При этом ни одна из систем не сочетает одновременно собственное развёртывание, открытый код, realtime-чат и независимую переиспользуемую базу задач с REST API. Это определяет нишу разрабатываемой системы. 1.5 Итоги анализа предметной области Выполнен анализ предметной области системы управления обучением. Сформированы функциональные и нефункциональные требования к серверной части, определены ограничения проекта. Рассмотрены четыре LMS- платформы – Moodle, Google Classroom, Stepik, Фоксфорд – и построена матрица отслеживаемости. Анализ показал, что ни одна из рассмотренных платформ не покрывает одновременно все ключевые требования проекта. Полученные результаты формируют основу для выбора технологий и построения архитектуры. 16 2 Выбор и обоснование технологий 2.1 Анализ архитектурных паттернов Архитектурный паттерн определяет способ организации кода, распределение ответственности и характер взаимодействия компонентов. От его выбора зависит тестируемость, сопровождаемость и расширяемость системы [1]. В данной главе рассматриваются три паттерна, применяемых при разработке серверной части: Domain-Driven Design DDD, Clean Architecture и Model-View-Controller MVC. 2.1.1 Domain-Driven Design Domain-Driven Design – подход, предложенный Эриком Эвансом, в котором ключевой единицей является модель предметной области [11]. Основные строительные блоки: – Сущности Entities – объекты с уникальным идентификатором и изменяемым состоянием. – Объекты-значения Value Objects – неизменяемые объекты, описываемые атрибутами. – Агрегаты – совокупности сущностей, управляемые как единое целое. – Репозитории – механизмы получения и хранения агрегатов. – Доменные сервисы – операции, не принадлежащие ни одной сущности. DDD обеспечивает точное соответствие программной модели бизнес- требованиям, однако требует активного участия экспертов предметной области и значительных трудозатрат на начальное моделирование. В типичном учебном проекте с одним исполнителем DDD избыточен: полноценная доменная модель оправдывает себя при сложных, постоянно меняющихся бизнес-правилах. 2.1.2 Clean Architecture Clean Architecture, предложенная Робертом Мартином [3], организует код в виде концентрических слоёв. Зависимости направлены только внутрь: внешние слои могут зависеть от внутренних, но не наоборот. Ключевые принципы: – Независимость от фреймворков – бизнес-логика не зависит от конкретного фреймворка. 17 – Тестируемость – бизнес-правила проверяются без запуска веб-сервера и базы данных. – Независимость от БД – замена СУБД не затрагивает бизнес-логику. В серверной части выделяются четыре слоя: 1 API Layer – приём запросов, валидация, маршрутизация. 2 Service Layer – реализация бизнес-сценариев. 3 Repository Layer – абстракция доступа к данным. 4 Models Layer – ORM-модели, конфигурация БД, внешние интеграции. Паттерн хорошо сочетается с REST-архитектурой: API-контроллеры образуют внешний слой, а бизнес-логика остаётся независимой от протокола. 2.1.3 MVC и MVT в контексте серверной разработки Model-View-Controller – один из наиболее известных архитектурных паттернов, исторически применяемых в веб-разработке. Приложение разделяется на три части: модель, отвечающую за данные и связанные с ними правила; представление, отвечающее за отображение информации; и контроллер, принимающий запрос и координирующий дальнейшую работу системы. Преимущество MVC заключается в простоте понимания и относительно невысоком пороге входа, особенно для приложений с выраженным CRUD-характером. Однако в серверной разработке на Python классический MVC в чистом виде встречается редко. Наиболее распространённый пример – фреймворк Django, который использует близкую, но не идентичную модель MVT Model-View-Template. В MVT слой Template отвечает за отображение данных, а слой View фактически совмещает функции контроллера и части прикладной логики. Это означает, что архитектурный подход в рамках Django по своей природе сильнее связан с самим фреймворком, чем в системах, построенных на FastAPI или Flask. С практической точки зрения различие между MVC и MVT связано не столько с формальной терминологией, сколько с характером зависимости прикладного кода от инфраструктуры. В MVC/MVT обработчики запросов обычно тесно связаны с жизненным циклом HTTP- 18 запроса, объектами фреймворка и механизмами ORM. При небольшом объёме функциональности это допустимо, однако по мере роста проекта затрудняет изоляцию бизнес-логики, повторное использование кода и тестирование независимо от web-слоя. Именно поэтому MVC и MVT удобны как стартовая модель для небольших веб-приложений, но менее удобны для системы с развитой серверной части и явное разделение ответственности. 2.2 Сравнение паттернов и обоснование выбора Для обоснования выбора необходимо рассмотреть, как каждый из трёх подходов проявляет себя в контексте разрабатываемой системы. Ниже дано сравнение по критериям, существенным для backend-проекта с одним разработчиком, REST API и требованием тестируемости. Итоговое сопоставление приведено в таблице 5. Тестируемость – важнейший критерий для проекта, где качество архитектуры нельзя подтвердить без автоматических тестов. Clean Architecture и DDD обеспечивают её естественным образом: сервисы и доменные объекты не зависят от фреймворка и БД, их можно тестировать без поднятия HTTP-сервера. В MVC и MVT контроллеры тесно связаны с фреймворком, и бизнес-логика нередко попадает прямо в них – это существенно усложняет написание изолированных тестов. Независимость от инфраструктуры – ключевое требование для систем, где технологический стек может меняться. Clean Architecture обеспечивает полную независимость бизнес-логики от фреймворка и СУБД через явные слои. DDD достигает схожего результата, но через агрегаты и доменные события. MVC/MVT жёстко привязан к фреймворку: замена Django на FastAPI потребует переписывания значительной части кода. Применимость к масштабу проекта – DDD оправдан при сложной предметной области с десятками агрегатов и постоянно меняющимися бизнес-правилами. В данном проекте предметная область достаточно прозрачна классы, уроки, задания, результаты, поэтому полноценная DDD-модель избыточна. Clean Architecture даёт чёткую структуру без лишней сложности. MVC/MVT – наиболее простой вход, но при росте системы она неизбежно начинает «протекать» в контроллеры. Таблица 5 – Сравнение архитектурных паттернов по ключевым критериям Критерий DDD Clean Architecture MVC / MVT Тестируемость Высокая: доменные объекты изолированы от инфраструктуры. Высокая: сервисы изолированы от инфраструктуры. Низкая: контроллеры связаны с фреймворком. Независимость от инфраструктуры Высокая: через порты и адаптеры. Высокая: зависимости направлены внутрь. Низкая: логика в Django views, Flask routes. Сложность внедрения Высокая: агрегаты, bounded contexts, ubiquitous language. Средняя: четыре слоя, правило задаёт сам язык. Низкая: структуру задаёт фреймворк. Применимость в проекте Избыточна: простая предметная область. Оптимальна: баланс структуры и сложности, подходит для проекта. Упрощённо: при росте проекта логика смешивается. Итоговое сравнение подтверждает выбор Clean Architecture: при средней сложности внедрения она даёт полную независимость бизнес-логики и высокую тестируемость, что соответствует требованиям проекта. 2.3 Обоснование технологического стека Python является одним из наиболее популярных языков для серверной веб-разработки благодаря выразительному синтаксису, богатой экосистеме и поддержке асинхронного программирования через asyncio [12]. Система аннотаций типов type hints обеспечивает статическую проверку кода без отказа от динамической природы языка [13]. В проекте используется версия 3.12+, что связано с тремя ключевыми факторами. Во-первых, Python 3.12 продолжает курс на повышение производительности интерпретатора, начатый в 3.11. Для серверного приложения это означает более быстрое выполнение вызовов функций, операций, работу с коллекциями и строками. Во-вторых, Python 3.12 улучшает работу системы типов и статического анализа. Для backend-разработчиков он важен, поскольку такие изменения упрощают описание типов и делает код более удобным для сопровождения. Для проекта, использующего FastAPI, Pydantic, SQLAlchemy и многослойную архитектуру, это особенно значимо: чем точнее описаны типы, тем надёжнее проверяется согласованность между схемами данных, сервисами и репозиториями в архитектуре. Дополнительным преимуществом является поддержка инструментов типизации и общее повышение качества работы с кодом. В-третьих, Python 3.12 представляет собой уже устоявшуюся и широко поддерживаемую версию. Её выбор позволяет опираться на актуальные версии FastAPI, Pydantic v2 и SQLAlchemy 2.0 без необходимости учитывать ограничения более старых интерпретаторов. По сравнению с Python 3.11 версия 3.12 обеспечивает работу в более производительной, предсказуемой и зрелой среде разработки. FastAPI – современный Python-фреймворк для создания REST API, один из наиболее популярных веб-фреймворков. Как показано на рисунке 1, популярность FastAPI стремительно растёт. Выбор FastAPI обусловлен следующим: фреймворк не диктует архитектуру, поддерживает асинхронность, встроенный Dependency Injection и автоматическую генерацию OpenAPI-документации. 21 Рисунок 1 – Сравнение популярности веб-фреймворков по данным Stack Overflow Developer Survey 2025 [14] PostgreSQL – реляционная СУБД с поддержкой ACID-транзакций, расширенных типов данных и развитой системы индексов [17,18]. PostgreSQL является отраслевым стандартом для веб-приложений среднего и крупного масштаба. SQLAlchemy 2.0 asyncio – зрелая ORM для Python с поддержкой асинхронного драйвера asyncpg [19]. Реализует паттерн Data Mapper, что соответствует принципам Clean Architecture [5]. Миграции управляются через Alembic [20]. React 18 + TypeScript – стек клиентской части [21]. Строгая типизация TypeScript обеспечивает предсказуемость кода; архитектура клиента построена на Feature-Sliced Design [22], что согласуется с разделением ответственности, принятым в Clean Architecture. 2.4 Итоги выбора технологий Рассмотрены три архитектурных паттерна – DDD, Clean Architecture, MVC/MVT. Показано, что выбор паттерна не зависит от фреймворка, так как FastAPI не ограничивает архитектуру. По результатам сравнения выбрана Clean Architecture, обеспечивающая оптимальное соотношение тестируемости и сложности. Определён технологический стек: Python, FastAPI, PostgreSQL, SQLAlchemy для серверной части и React + TypeScript для клиентской. 22 3 Разработка архитектуры приложения 3.1 Слои архитектуры Серверная часть организована в четыре слоя с однонаправленными зависимостями [3,23]: – API Layer – приём HTTP-запросов, валидация через Pydantic, делегирование сервисному слою. – Service Layer – слой бизнес-логики; сервисы используют репозитории для доступа к данным. – Repository Layer – слой доступа к данным; Repository скрывает детали SQLAlchemy от сервисного слоя. – Models Layer – ORM-модели, конфигурация базы данных, миграции, внешние интеграции Redis для realtime-чата. Дополнительно выделены Pydantic-схемы schemas для валидации входных данных и формирования ответов, а также core: конфигурация, JWT, хеширование. Эти блоки находятся вне основных слоёв и не нарушают правило зависимостей. Структура директорий: app/ ├── api/ API Layer: роутеры, DI, обработчики ошибок │ ├── v1/ эндпоинты │ ├── http_errors.py фабрики HTTPException │ └── errors.py ├── services/ Service Layer: бизнес-сценарии ├── repositories/ Repository Layer: SQL-запросы через ORM ├── models/ Models Layer: ORM-модели SQLAlchemy ├── schemas/ Pydantic DTO ├── core/ конфигурация, безопасность, утилиты └── db/ сессии AsyncSession и URL-сервера ├── realtime/ WebSocket-чат и Redis Pub/Sub └── scripts/ утилиты: seed, export OpenAPI 23 3.2 Dependency Injection как механизм связывания слоёв Связывание слоёв реализовано через встроенный механизм Dependency Injection через Depends. Каждый эндпоинт получает нужный сервис через зависимость. Схема DI для слоя аутентификации приведена в листинге 1. Листинг 1 – Dependency Injection для сервиса аутентификации app/api/dependencies.py def get_auth_service db: AsyncSession = Dependsdb_session.get_db, -> AuthService: return AuthServicedb async def get_current_user token: TokenContext = Dependsget_current_user_id, db: AsyncSession = Dependsdb_session.get_db, -> User: user = await UserRepositorydb.get_by_idtoken.user_id if not user: raise HTTPExceptionstatus_code=401, detail={"code": "user_not_found"} if token.role != user.role: raise HTTPException status_code=403, detail={"code": "role_changed", "message": "Re-authenticate"} return user Такая схема обеспечивает слабую связанность: в тестах любая зависимость подменяется через dependency_overrides. Зависимости get_current_teacher, get_current_student встраивают проверку роли в DI-цепочку, исключая дублирование в каждом эндпоинте. 3.3 Обработка ошибок В архитектуре принят следующий подход к ошибкам: – Lookup-методы сервисов возвращают None при отсутствии сущности. Маршрутизатор проверяет результат и явно формирует ответ. – Команды сервисов создание, обновления, оркестрация с несколькими проверками порождают ServiceErrormessage, code внутри сервисного слоя. Код ошибки несёт семантику, а HTTP-статус определяет обработчик эндпоинта. – Необработанные исключения, не перехваченные явно, приводят к ответу 500 Internal Server Error – индикатор ошибки в коде сервера. В модуле app/api/http_errors.py определены фабрики not_found, forbidden, bad_request и from_service_error, которые строят HTTPException с унифицированным телом ответа {"error": {"code", "message"}, "request_id": ...}. Глобальный обработчик app/api/errors.py обеспечивает единообразный формат ошибок. 3.4 Диаграмма компонентов На рисунке 2 представлена компонентная модель серверной части. Диаграмма показывает, каким образом взаимодействуют основные компоненты системы, отражающая Clean Architecture. Центральное место в диаграмме занимают сервисы, реализующие основные сценарии использования системы: регистрация и аутентификация пользователей, управление классами, уроками и заданиями, работа с базой задач, автоматическая проверка ответов, расчёт прогресса и работа с теоретическими материалами. Выделение данных сценариев в отдельные сервисы позволяет изолировать бизнес-логику и исключить её дублирование в обработчиках HTTP-запросов. Такое разделение необходимо для того, чтобы сервисный слой не зависел от SQL-запросов, транзакций и ORM-механизмов хранения данных и повышается тестируемость. Отдельно выделена подсистема чата, обеспечивающая обмен сообщениями. Она включает обычный REST-интерфейс и WebSocket-канал для доставки событий в режиме реального времени через Redis Pub/Sub, используемый для горизонтального масштабирования, сохраняя при этом единые принципы работы с сущностями, авторизацией и хранением истории сообщений. Таким образом, компонентная диаграмма демонстрирует, что серверная часть построена как согласованная система взаимосвязанных подсистем, в которой каждый компонент выполняет строго определённую функцию: приём запроса, реализация прикладного сценария, доступ к данным или обмен сообщениями в реальном времени. Также распределение ответственности соответствует принципам Clean Architecture и обеспечивает сопровождаемость разрабатываемого программного обеспечения. 3.5 Проектирование базы данных Спроектирована реляционная схема базы данных, охватывающая все ключевые сущности предметной области. ER-диаграмма представлена на рисунке 3. Рисунок 3 – ER-диаграмма данных системы управления обучением 27 Основные сущности: – users – профиль пользователя логин, email, ФИО, активная роль. – login_data – хеш пароля, изолированный от профиля для разграничения доступа. – teachers / students – профили ролей; один пользователь может иметь оба, переключая активную роль. – classrooms – учебный класс с кодом приглашения. – student_classroom – членство студента в классе «многие ко многим». – lessons – урок, привязанный к классу. – homework – домашнее задание, привязанное к уроку. – problems – независимая база задач, переиспользуемая в разных домашних заданиях. – homework_problem – связь задания и задачи с атрибутом «баллы». – statistics – результат выполнения задания студентом. – theme / theory – иерархия тем и теоретические материалы. – chat / message – чат класса и его сообщения. Пример ORM-модели приведён в листинге 2. Ключевое решение – отделение users от login_data: это изолирует хеш пароля от обычных выборок профиля. 28 Листинг 2 – ORM-модель пользователя SQLAlchemy app/models/users.py class UserBase: __tablename__ = "users" id = ColumnInteger, primary_key=True, index=True username = ColumnString100, unique=True, nullable=False, index=True email = ColumnString255, unique=True, nullable=False, index=True first_name = ColumnString100, nullable=False last_name = ColumnString100, nullable=False full_name = ColumnString255, nullable=False role = ColumnString20, nullable=False, default="student" is_active = ColumnBoolean, default=True, nullable=False created_at = ColumnDateTimetimezone=True, default=utc_now, nullable=False login_data = relationship"LoginData", uselist=False, cascade="all, delete- orphan" teacher = relationship"Teacher", uselist=False, cascade="all, delete- orphan" student = relationship"Student", uselist=False, cascade="all, delete- orphan" Управление миграциями выполняется через Alembic: каждое изменение схемы оформляется в виде пронумерованной ревизии [20]. 3.6 Результаты проектирования Разработана детальная архитектура серверной части на основе Clean Architecture. Описаны четыре слоя и их зоны ответственности, показана структура директорий. Обоснован подход к обработке ошибок: сервисы возвращают None для lookup-операций и поднимают ServiceError для команд; HTTP-семантику определяют маршрутизаторы. Описан механизм Dependency Injection, обеспечивающий слабую связанность и тестируемость. Спроектирована реляционная схема базы данных, охватывающая все ключевые сущности предметной области. Разработанная архитектура является основой для реализации серверной части в следующей главе. 29 4 Разработка серверной части интернет-ресурса 4.1 Реализация REST API на FastAPI Серверная часть реализована на Python версии 3.12+ с использованием FastAPI [15,16]. Все маршруты сгруппированы по функциональным областям и зарегистрированы под единым корневым роутером в app/api/v1/__init__.py: /api/v1/auth/ – аутентификация и управление ролями /api/v1/classrooms/ – классы, приглашения, чат, WebSocket /api/v1/lessons/ – уроки /api/v1/homework/ – домашние задания /api/v1/problems/ – база задач /api/v1/testing/ – приём ответов, автопроверка /api/v1/statistics/ – прогресс и статистика /api/v1/theory/ – теоретические материалы При проектировании API соблюдаются стандартные принципы REST [24]. Взаимодействие строится вокруг ресурсов, доступ к которым осуществляется через стандартные HTTP-методы GET, POST, PATCH, DELETE. Принцип взаимодействия между клиентом и сервером посредством REST API проиллюстрирован на рисунке 4. Рисунок 4 – Принцип взаимодействия через REST API Каждый обработчик выполняет единственную задачу: валидирует входные данные через Pydantic-схему, вызывает сервис и транслирует результат в HTTP-ответ. Пример эндпоинта аутентификации приведён в листинге 3. 30 Листинг 3 – Реализация эндпоинта аутентификации app/api/v1/auth.py @router.post"/login", response_model=TokenResponse async def login credentials: LoginRequest, auth_service: AuthService = Dependsdeps.get_auth_service, -> TokenResponse: try: token = await auth_service.authenticatecredentials except ServiceError as exc: if exc.code == "inactive_user": raise http_errors.forbiddenexc.message, code=exc.code raise if token is None: raise http_errors.unauthorized"Invalid username/email or password" return token Листинг демонстрирует принцип явной обработки ошибок: обработчик знает семантику каждого кода и выбирает HTTP-статус самостоятельно. Незнакомое исключение не перехватывается – ответ 500 сигнализирует об ошибке на стороне сервера. Практическая ценность такого подхода заключается в том, что поведение API становится предсказуемым как для разработчика серверной части, так и для разработчика клиента. Для каждой типовой ситуации – неверные учётные данные, деактивированная учётная запись, отсутствие ресурса или нарушение ограничений доступа – обработчик формирует понятный и однозначный ответ. Это упрощает сопровождение системы и исключает скрытые преобразования ошибок, затрудняющие отладку. 4.2 Реализация сервисного слоя Сервисный слой реализует бизнес-сценарии и не зависит от HTTP- контекста. Сервисы используют репозитории для доступа к данным, а все проверки правил приложения выполняются до передачи управления на уровень ORM. Такой подход обеспечивает воспроизводимость бизнес-правил независимо от того, каким именно способом был инициирован вызов: через REST API, WebSocket или внутренний скрипт. Ключевой фрагмент сервиса аутентификации приведён в листинге 4. 31 Листинг 4 – Ключевой фрагмент сервиса аутентификации app/services/auth.py – метод authenticate async def authenticate self, login: LoginRequest, -> TokenResponse | None: user = await self.user_repo.get_by_username_or_email login.username_or_email if not user: return None if not user.is_active: raise ServiceError"User account is inactive", code="inactive_user" info = await self.login_repo.get_by_user_iduser.id if not info or not security.verify_password login.password, info.hashed_password: return None token = security.create_access_token {"sub": struser.id, "role": user.role} return TokenResponse access_token=token, user=UserResponse.model_validateuser, Сервис TestingService реализует ключевой сценарий автоматической проверки ответов студента. Для системы управления обучением данный сервис имеет принципиальное значение, поскольку именно он обеспечивает прикладную ценность платформы: ответ студента не просто сохраняется, а сразу сравнивается с эталоном, после чего обновляются накопленный балл, число попыток и затраченное время. Код соответствующего фрагмента приведён в листинге 5. 32 Листинг 5 – Фрагмент реализации автоматической проверки ответа app/services/testing.py – фрагмент метода submit_answer async def submit_answer self, answer_data: AnswerSubmit, student_user_id: int, -> StatisticsResponse: ... [Проверка права доступа, существования задачи и её принадлежности к ДЗ] ... stats = await self.stats_repo.get_or_createstudent.id, problem.id, homework.id Автоматическая проверка ответа студента is_correct = self._check_answeranswer_data.answer, problem.answer Начисление баллов с ограничением по максимальному баллу new_score = minstats.score + hp.problem_points, stats.max_score if is_correct else stats.score Фиксация попытки и затраченного времени updated = await self.stats_repo.updatestats.id, { "score": new_score, "status": "in_progress", "attempts_count": stats.attempts_count + 1, "time_spent_minutes": stats.time_spent_minutes + answer_data.time_spent_minutes},) return StatisticsResponse.model_validateupdated def _check_answerself, student: str, correct: str | None -> bool: if not correct: return False return student.strip.lower == correct.strip.lower 4.3 Слой безопасности Безопасность реализована на двух уровнях: хеширование паролей и аутентификация через JWT. Пароли хешируются алгоритмом Argon2id – устойчивым к атакам с использованием GPU и ASIC, рекомендованным для хранения учётных данных [25]. JWT-токены формируются с краткими сроками действия и содержат 33 идентификатор пользователя и активную роль, что позволяет обрабатывать каждую просьбу без обращения к базе данных. Выбор Argon2id обусловлен криптографическими свойствами алгоритма. В отличие от простых хеш-функций, он учитывает не только число итераций, но и объём используемой памяти, что повышает стоимость атак перебором. Применение JWT, в свою очередь, позволяет реализовать безсессионную аутентификацию и уменьшить нагрузку на сервер при каждом запросе, сохраняя возможность централизованной проверки роли и срока действия токена. Листинг 6 – Реализация безопасности: Argon2id и JWT app/core/security.py _MEMORY_COST_KIB = 1024 * 64 64 MB ph = argon2.PasswordHasher time_cost=2, memory_cost=_MEMORY_COST_KIB, parallelism=1, hash_len=32, salt_len=16, def verify_passwordplain: str, hashed: str -> bool: try: return ph.verifyhashed, plain return True except argon2.exceptions.VerifyMismatchError: return False def create_access_tokendata: dict -> str: payload = {**data, "exp": utc_now + timedeltaminutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES} return jose_jwt.encodepayload, settings.SECRET_KEY, algorithm=settings.ALGORITHM 4.4 Управление конфигурацией и запуск Конфигурация приложения хранится в переменных окружения, загружаемых через pydantic-settings [26]. Такой способ конфигурирования обеспечивает независимость параметров запуска от исходного кода и соответствует общепринятой практике построения серверных приложений. Точка входа app/main.py подключает middleware CORS, 34 RequestIdMiddleware, глобальные обработчики ошибок и API-роутер с префиксом /api/v1. При наличии REDIS_URL в момент старта инициализируется Redis Pub/Sub-брокер для многоэкземплярного WebSocket- чата; без него приложение стартует без realtime-компонента. Серверная часть автоматически формирует OpenAPI-спецификацию, которая используется как машинно-читаемое описание контракта API. Это важно не только для ручной проверки эндпоинтов, но и для интеграции с клиентской частью, поскольку OpenAPI-описание может выступать основой для кодогенерации, тестирования и валидации совместимости версий интерфейса. На рисунке 5 показан интерфейс OpenAPI-документации. Рисунок 5 – Интерфейс автоматически сформированной OpenAPI- документации 4.5 Тестирование Принципы Clean Architecture обеспечивают практическую независимость сервисного слоя от веб-слоя. Тесты используют сервисы напрямую, без поднятия FastAPI. Для каждого теста создаётся отдельная схема PostgreSQL, которая накатывает таблицы из метаданных SQLAlchemy и удаляется по завершении. Это позволяет проверить бизнес-логику в реалистичных условиях при полной изоляции между тестами [19]. В составе проекта реализовано 35 тестов для AuthService, TestingService, ChatService, ProblemService, ResultService, TheoryService, а также для модулей realtime. Автоматизированное тестирование выполняется с помощью pytest и направлено на проверку корректности реализованных сценариев. Тесты охватывают регистрацию и аутентификацию, автоматическую проверку ответов, подсистему обмена сообщениями, OpenAPI-контракт, расчёт прогресса и работу realtime-компонентов. На рисунке 6 показан результат выполнения набора тестов серверной части. Рисунок 6 – Автоматизированные тесты серверной части 4.6 Клиентское представление Клиентская часть платформы реализована на React 18 с TypeScript [21]. Архитектура фронтенда построена по методологии Feature-Sliced Design FSD [22], которая структурирует код в шесть слоёв по убыванию области ответственности. Принципиальная схема данной методологии представлена на рисунке 7. 36 Рисунок 7 – Схема слоёв методологии Feature-Sliced Design В рамках проекта слои распределены следующим образом: src/ ├── app/ – инициализация, провайдеры, роутинг ├── pages/ – страницы Login, Classroom, Lesson, ... ├── widgets/ – составные блоки Header, Sidebar, ChatPanel ├── features/ – сценарии пользователя JoinClassroom, SubmitAnswer, ... ├── entities/ – доменные данные Classroom, Lesson, User, Message └── shared/ – переиспользуемые утилиты, UI-kit, HTTP-клиент Взаимодействие с сервером построено через axios-клиент в shared/api/ axios.ts. Серверный контракт описан в файле openapi.json, генерируемом бэкенд-командой python -m app.scripts.export_openapi. Управление серверным состоянием реализовано через TanStack Query v4, локальное состояние – через Zustand. Формы валидируются с помощью react-hook-form и zod. UI построен на примитивах Radix UI и стилизован через Tailwind CSS. Требование межстраничной навигации реализуется за счёт клиентского роутинга в app слое, где определяются основные пользовательские переходы между экранами входа, списком классов, страницами уроков, домашними заданиями и представлением результатов. Такое построение навигации позволяет сохранить целостность пользовательского сценария: преподаватель переходит от списка учебных классов к конкретному уроку и связанным с ним заданиям, а студент – от структуры курса к выполнению конкретного домашнего задания и просмотру результата проверки. 37 Требование современного веб-дизайна обеспечивается за счёт использования единой компонентной базы и общей системы стилизации. Применение Radix UI и Tailwind CSS позволяет поддерживать единообразие визуальных элементов, адаптивность интерфейса и предсказуемость поведения экранов, что особенно важно для образовательной платформы, ориентированной на ежедневное использование преподавателями и студентами. Подобная организация клиентской части важна с точки зрения backend- разработки, поскольку позволяет выстроить устойчивый канал взаимодействия между сервером и пользовательским интерфейсом. Чёткое разделение страниц, сущностей и пользовательских сценариев снижает риск рассогласования между моделью данных на сервере и представлением этих данных на клиенте. Наличие единого OpenAPI-контракта дополнительно упрощает синхронизацию обеих частей системы. Интерфейс класса преподавателя должен решать две практические задачи: быстро показывать состав учебной группы и давать доступ к операциям управления ею. На рисунке 8 показан интерфейс, в котором преподаватель может видеть список учеников, их успеваемость, код приглашения и средства навигации, что позволяет быстро ориентироваться в текущем состоянии учебного процесса. 38 Рисунок 8 – Главная страница класса преподавателя Страница выполнения задания для студента должна концентрировать внимание на решении задач, отображать подробное условие и предоставлять удобную возможность для ввода ответа. На рисунке 9 показан интерфейс, в котором студент может перемещаться между заданиями и вводить ответы. 39 Рисунок 9 – Страница выполнения домашнего задания для студента Связь фронтенда с бэкендом реализована через единый API-контракт OpenAPI: изменение серверного эндпоинта отражается в спецификации и может быть подхвачено кодогенерацией на стороне клиента, что обеспечивает согласованность интерфейсов без ручной синхронизации. 4.7 Итоги разработки Разработана серверная часть на Python с использованием FastAPI, PostgreSQL и SQLAlchemy. REST API включает девять групп эндпоинтов, покрывающих все функциональные требования системы. Обработка ошибок построена явно: сервисы возвращают None для lookup-операций и поднимают ServiceError для командных сценариев; HTTP-статус выбирает маршрутизатор. Обеспечена безопасность: Argon2id для хранения паролей и JWT для аутентификации. Тестируемость архитектуры подтверждена набором автоматизированных тестов, работающих на уровне сервисного слоя без HTTP. Реализована клиентская часть на React + TypeScript по методологии Feature- Sliced Design, потребляющая REST API через OpenAPI-контракт. 40 ЗАКЛЮЧЕНИЕ В ходе выполнения курсовой работы разработана серверная часть веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture. На первом этапе выполнен анализ предметной области системы управления обучением. Сформированы функциональные и нефункциональные требования к серверной части. Проведён обзор четырёх LMS-платформ – Moodle, Google Classroom, Stepik и Фоксфорд – и построена матрица отслеживаемости, выявившая нишу, не закрытую ни одним аналогом: собственное развёртывание, открытый код, независимая база задач, встроенный realtime-чат, полноценный REST API с возможностью развёртывания на собственном сервере. На втором этапе проведён сравнительный анализ архитектурных паттернов DDD, Clean Architecture и MVC/MVT. Показано, что для разрабатываемой системы наиболее целесообразна Clean Architecture, поскольку она обеспечивает явное разделение ответственности, высокую тестируемость и независимость прикладной логики от инфраструктурных компонентов. Сформирован технологический стек: Python 3.12+, FastAPI, PostgreSQL, SQLAlchemy. На третьем этапе разработана детальная архитектура приложения. Описаны слои API, Service, Repository и Models, механизм Dependency Injection и принятый подход к обработке ошибок. Спроектирована реляционная схема базы данных, охватывающая все сущности предметной области. На четвёртом этапе реализованы серверная и клиентская части. Серверная часть включает девять групп REST-эндпоинтов, сервисный слой с автоматической проверкой ответов, слой безопасности Argon2id, JWT и WebSocket-чат с опциональным Redis Pub/Sub-брокером. Клиентская часть реализована на React 18 + TypeScript по методологии Feature- Sliced Design. Корректность сервисного слоя подтверждена набором автоматизированных тестов, работающих без HTTP- слоя. 41 Полученные результаты подтверждают применимость Clean Architecture и выбранного стека для построения серверной части систем управления обучением: архитектура допускает дальнейшее расширение функциональности и масштабирование без переписывания ядра системы. 42 СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 1. Минакова О.В. Программная инженерия: учебник для вузов [Электронный ресурс]: учебник для вузов ISBN 978-5-507-49278-7. Санкт-Петербург: Лань, 2024. 212 с. URL: https://e.lanbook.com/ book/414989 дата обращения: 11.05.2026. 2. Rosário A.T., Dias J.C. Learning Management Systems in Education: Research and Challenges. DOI: 10.4018/978-1-6684-4706-2.ch003 // Digital Active Methodologies for Educative Learning Management / под ред. Geada N., Jamil G.L. Hershey, PA: IGI Global, 2022. С. 47–77. 3. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения: ISBN 978-5-4461-0903-1. 2-е изд. Санкт-Петербург: Питер, 2021. 352 с. 4. Киселева Т.В. Программная инженерия [Электронный ресурс]: учебное пособие. Ставрополь: СКФУ, 2017. Ч. 2. 100 с. URL: https://e.lanbook.com/ book/155149 дата обращения: 11.05.2026. 5. Туманова М.Б., Михайлова Е.К., Муравьева Е.А. Проектирование программных систем [Электронный ресурс]: учебное пособие ISBN 978-5-7339-2050-4. Москва: РТУ МИРЭА, 2023. 138 с. URL: https://e.lanbook.com/book/398273 дата обращения: 11.05.2026. 6. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. Москва: Стандартинформ, 2017. 7. Moodle – Open-source learning platform [Электронный ресурс]. URL: https://moodle.org/ дата обращения: 11.05.2026. 8. Google Classroom – официальный сайт [Электронный ресурс]. URL: https://edu.google.com/intl/ru/ workspace-for-education/ classroom/ дата обращения: 11.05.2026. 9. Stepik – онлайн-платформа для обучения [Электронный ресурс]. URL: https://stepik.org/ дата обращения: 11.05.2026. 10. Фоксфорд – онлайн-школа для учеников 1–11 классов [Электронный ресурс]. URL: https:// foxford.ru/ дата обращения: 11.05.2026. 11. Персиваль Г., Грегори Б. Паттерны разработки на Python: TDD, DDD и событийно-ориентированная архитектура: ISBN 978-5-4461-1969-7. Санкт-Петербург: Питер, 2022. 336 с. 12. Python 3 Documentation [Электронный ресурс]. URL: https://docs.python. org/3/ дата обращения: 11.05.2026. 13. Баланов А.Н. Комплексное руководство по разработке: от мобильных приложений до веб-технологий [Электронный ресурс]: учебное пособие ISBN 978-5-507-53193-6. 2-е изд. Санкт-Петербург: Лань, 2025. 412 с. URL: https://e.lanbook.com/book/478178 дата обращения: 11.05.2026. 14. Stack Overflow Developer Survey 2025 [Электронный ресурс]. URL: https://survey.stackoverflow.co/2025/ дата обращения: 11.05.2026. 15. Елисеев А.И., Минин Ю.В. Разработка программных интерфейсов веб-приложений с использованием фреймворка FastAPI [Электронный ресурс]: учебное пособие ISBN 978-5-8265-2821-1. Тамбов: ТГТУ, 2024. 81 с. URL: https://e.lanbook.com/book/472310 дата обращения: 11.05.2026. 16. Ramírez S. FastAPI – официальная документация [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ дата обращения: 11.05.2026. 17. PostgreSQL Documentation [Электронный ресурс]. URL: https://www. postgresql.org/docs/ дата обращения: 11.05.2026. 18. Волк В.К., Осеев В.Ю., Черепанов О.С. Базы данных [Электронный ресурс]: учебник ISBN 978-5-9729-2594-0. Вологда: Инфра-Инженерия, 2025. 544 с. URL: https://e.lanbook.com/book/499772 дата обращения: 11.05.2026. 19. SQLAlchemy 2.0 Documentation [Электронный ресурс]. URL: https://docs. sqlalchemy.org/en/20/ дата обращения: 11.05.2026. 20. Alembic – официальная документация [Электронный ресурс]. URL: https://alembic.sqlalchemy.org/ дата обращения: 11.05.2026. 21. React Documentation [Электронный ресурс]. URL: https://react.dev/ дата обращения: 11.05.2026. 22. Feature-Sliced Design: официальная документация [Электронный ресурс]. URL: https://feature-sliced.github.io/documentation/ru/ дата обращения: 11.05.2026. 23. Водяхо А.И. и др. Архитектурные решения информационных систем [Электронный ресурс]: учебник для вузов ISBN 978-5-507-44710-7. 3-е изд. Санкт-Петербург: Лань, 2022. 356 с. URL: https://e.lanbook.com/book/ 254624 дата обращения: 11.05.2026. 24. Аллатов А.Н. Интерфейсы прикладного программирования [Электронный ресурс]: учебное пособие ISBN 978-5-7339-2342-0. Москва: РТУ МИРЭА, 2024. 157 с. URL: https://e.lanbook.com/book/457043 дата обращения: 11.05.2026. 25. Хоффман Э. Безопасность веб-приложений: ISBN 978-5-4461-1706-8. Санкт-Петербург: Питер, 2021. 336 с. 26. Pydantic V2 Documentation [Электронный ресурс]. URL: https://docs. pydantic.dev/latest/ дата обращения: 11.05.2026. 45